



**XTART**  
FORMACIÓN PROFESIONAL

## **LOVECODE**

### **Memoria técnica 3ª Evaluación**

**Pablo Pedrón López**

1º DAW Curso 25-26

XTART – VALENCIA

Módulos:

Bases de datos, Entornos de desarrollo, Lenguaje de marcas, Programación,  
Proyecto intermodular, Sistemas Informáticos.

Profesorado:

José Luis López De Pablo, Esteban Castañer, Pedro Campos.

Fecha de entrega:

31 de mayo de 2026

# ÍNDICE

## Contenido

|                                               |    |
|-----------------------------------------------|----|
| Contenido .....                               | 2  |
| 1. INTRODUCCIÓN.....                          | 4  |
| 2. DESCRIPCIÓN GENERAL DEL SISTEMA .....      | 4  |
| 3. SISTEMAS INFORMÁTICOS .....                | 5  |
| 3.1 CONEXIONADO .....                         | 5  |
| 3.2 AUTOMATIZACIÓN DEL SISTEMA.....           | 5  |
| 3.2.1 ARRANQUE DEL SISTEMA.....               | 5  |
| 3.2.4. BACKUP MANUAL .....                    | 6  |
| 3.2.3 BACKUP TERMINAL.....                    | 7  |
| 3.2.4 SCRIPT AUTOMÁTICO BACKUP .....          | 7  |
| 3.2.5 RESTAURACIÓN DEL BACKUP .....           | 9  |
| 4. BASES DE DATOS.....                        | 9  |
| 4.1 DIAGRAMA ENTIDAD-RELACIÓN .....           | 9  |
| 4.2 Triggers (Disparadores) .....             | 13 |
| 4.3 PROCEDIMIENTOS ALMACENADOS .....          | 13 |
| 4.4 USUARIOS .....                            | 13 |
| 4.5 CONSULTAS SQL Y EVIDENCIAS.....           | 15 |
| 5. PROGRAMACIÓN .....                         | 19 |
| 5.1 CONEXIÓN CON LA BASE DE DATOS .....       | 19 |
| 5.2 OPERACIONES QUE REALIZA EL SERVIDOR ..... | 20 |
| 5.3 REGISTRO Y LOGIN.....                     | 20 |
| 5.4 ALMACENAMIENTO DE LIKES.....              | 21 |
| 5.5 EVIDENCIAS DE FUNCIONAMIENTO .....        | 22 |
| 6. LENGUAJE DE MARCAS.....                    | 24 |
| 6.1 ESTRUCTURA DEL FRONTEND .....             | 24 |
| 6.2 TECNOLOGÍAS.....                          | 25 |
| 7. ENTORNOS DE DESARROLLO .....               | 26 |
| 7.1 CONTROL DE VERSIONES.....                 | 26 |
| 7.2 SOFTWARE UTILIZADO .....                  | 27 |

## Memoria técnica Proyecto Intermodular

|                                            |    |
|--------------------------------------------|----|
| 8. FUNCIONAMIENTO DEL PROYECTO.....        | 27 |
| 9. PROBLEMAS ENCONTRADOS Y SOLUCIONES..... | 29 |
| 10. CONCLUSIONES .....                     | 30 |
| 11. BIBLIOGRAFÍA Y WEBGRAFÍA.....          | 30 |

## 1. INTRODUCCIÓN

LoveCode es una aplicación web para conocer personas afines a ti, diseñada específicamente para la comunidad de desarrolladores y programadores. La plataforma permite a los usuarios crear perfiles, explorar otros perfiles de programadores, dar likes y generar matches, replicando la experiencia de una app de citas pero con una estética y lenguaje propios del mundo del desarrollo de software.

El proyecto ha sido desarrollado como trabajo de la 3ª evaluación, integrando los conocimientos que hemos aprendido durante el curso de frontend, backend en Java, diseño y gestión de bases de datos relacionales con MariaDB (Con sus usuarios y permisos) y el conexionado a una máquina virtual en la que reside la base de datos.

## 2. DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema implementa una arquitectura web de tres capas principales conectadas entre sí:

1. **Capa de Almacenamiento y Persistencia (Base de Datos):** Gestionada a través de MariaDB sobre un servidor Linux Debian. Centraliza el almacenamiento de usuarios, interacciones (Likes y Matches) y el catálogo de tecnologías. Además, la base de datos incluye procedimientos almacenados para llamarlas directamente desde el backend y dos triggers encargados de gestionar acciones automáticas dentro del sistema.
2. **Capa de Lógica de Negocio (Backend Server):** Está desarrollada en Java utilizando Spring Boot y Maven. Se encarga de gestionar la lógica de la aplicación, conectar con la base de datos y enviar la información al frontend mediante JSON.
3. **Capa de Interfaz de Usuario (Frontend Web):** Está desarrollada utilizando HTML, CSS y JavaScript. Esta parte es la encargada de mostrar toda la interfaz de la aplicación y permitir que los usuarios interactúen con el sistema de forma visual. Incluye formularios de registro e inicio de sesión, visualización de perfiles y el sistema de likes y matches entre usuarios. Además, JavaScript permite la comunicación con el backend para enviar y recibir información en tiempo real.

### 3. SISTEMAS INFORMÁTICOS

#### 3.1 CONEXIONADO

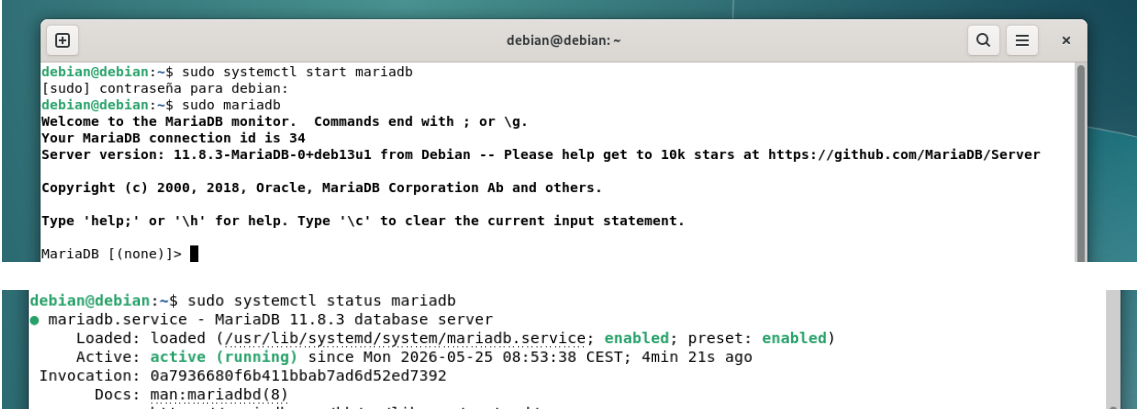
La arquitectura de red del proyecto se apoya en una máquina virtual configurada en modo Host-Only, lo que crea un entorno de comunicación privado y seguro entre el sistema anfitrión (Windows) y el sistema invitado (Debian). Con el objetivo de permitir la conexión de herramientas externas, como MySQL Workbench y la propia aplicación Java, al servidor de base de datos MariaDB, ha sido necesario realizar varios ajustes clave en la configuración.

En primer lugar, se modificó el parámetro bind-address dentro del archivo de configuración de MariaDB (/etc/mysql/mariadb.conf.d/50-server.cnf), pasando de 127.0.0.1 a 0.0.0.0, lo que permite que el servicio escuche solicitudes entrantes desde cualquier interfaz de red.

Además, se configuró el firewall de Debian (ufw) para habilitar el puerto 3306 y se concedieron permisos de acceso remoto al usuario administrador, asegurando así una conexión estable y continua entre los distintos componentes del sistema.

#### 3.2 AUTOMATIZACIÓN DEL SISTEMA

##### 3.2.1 ARRANQUE DEL SISTEMA



```

debian@debian:~$ sudo systemctl start mariadb
[sudo] contraseña para debian:
debian@debian:~$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>

debian@debian:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.3 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-25 08:53:38 CEST; 4min 21s ago
   Invocation: 0a7936680f6b411bbab7ad6d52ed7392
   Docs: man:mariadb(8)
         https://mariadb.com/kb/en/library/systemd/

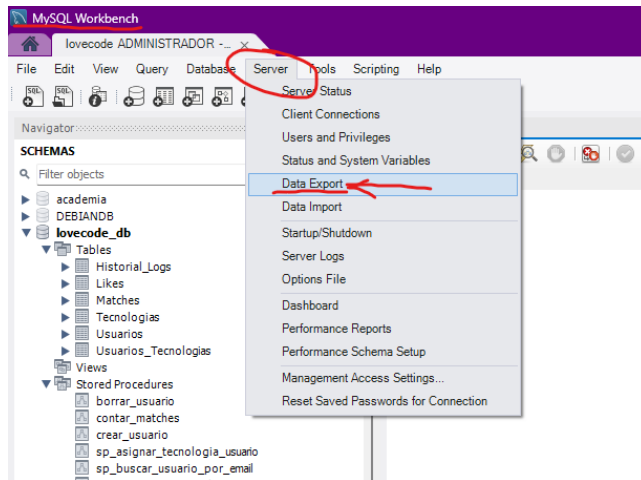
```

- **Iniciar el servicio:** `sudo systemctl start mariadb`
- **Acceso local (Nativo):** `sudo mariadb -u administrador -p lovecode_db`
- **Comprobar estado:** `sudo systemctl status mariadb`

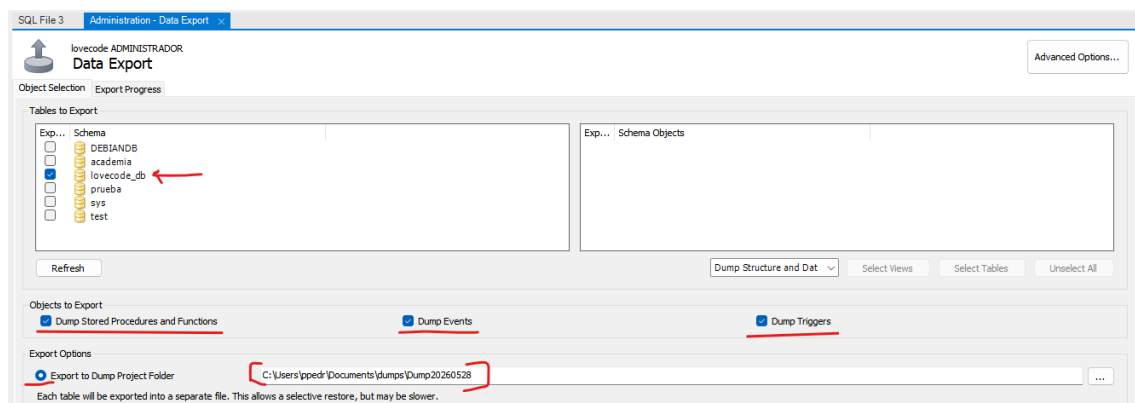
**Modo enabled:** Configurar el servicio en modo enabled (mediante `sudo systemctl enable mariadb`) asegura que el servidor de bases de datos se levante de forma automática durante el arranque de la máquina virtual Debian. Esto garantiza que las tablas, los datos y toda la lógica de los triggers estén disponibles inmediatamente para la API de Spring Boot en cuanto se enciende el servidor.

### 3.2.4. BACKUP MANUAL

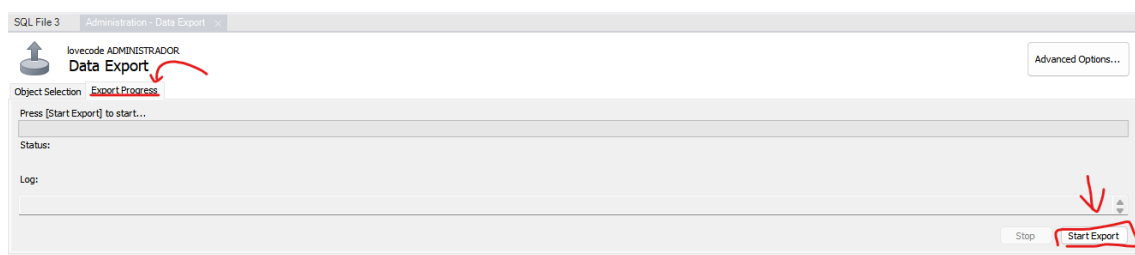
Para n el menú superior, debemos hacer click en Server y seleccionar Data Export.



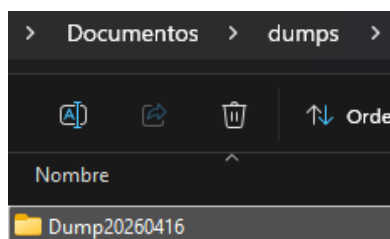
Marcamos la base de datos (lovecode\_db), marcamos las casillas para incluir *Stored Procedures*, *Events* y *Triggers*, y elegimos la ruta de guardado:



Cambiamos a la pestaña Export Progress (arriba a la izquierda) y le damos al botón Start Export (abajo a la derecha).



Se creará la carpeta con el backup en el sitio especificado:



### 3.2.3 BACKUP TERMINAL

```
sudo mariadb-dump -p --routines lovecode_db > BackUp_LoveCode_db_25-05-2026.sql
```

- **sudo:** Ejecuta la utilidad con privilegios de superusuario del sistema para asegurar el acceso correcto a las herramientas de la base de datos.
- **mariadb-dump:** Efectúa un volcado lógico, transformando el esquema y los datos en un conjunto de sentencias SQL.
- **-p:** Indica al sistema que solicite la contraseña de acceso de forma interactiva y oculta, protegiendo las credenciales.
- **--routines:** Por defecto, las copias de seguridad de MariaDB no incluyen las funciones ni los procedimientos almacenados (Stored Procedures), si queremos que se guarden es necesario ponerlo.
- **lovecode\_db:** Especifica la base de datos origen de la cual se va a extraer toda la información.
- **>:** Operador de redirección de salida de Linux.
- **BackUp\_LoveCode\_db\_25-05-26.sql:** Nombre asignado al archivo final.

### 3.2.4 SCRIPT AUTOMÁTICO BACKUP

```
$IP = "192.168.204.129"
$Carpeta = "C:\Backups_Debian"
$Archivo = "$Carpeta\backup_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').sql"

# 1. Comprueba la conexión con la máquina virtual (si falla, detiene el script)
if (!(Test-Connection -ComputerName $IP -Count 1 -Quiet)) { exit }

# 2. Crea la carpeta de backups si no existe
if (!(Test-Path $Carpeta)) { New-Item -ItemType Directory -Path $Carpeta | Out-Null }

# 3. Ejecuta el backup y lo guarda en el archivo con fecha y hora
ssh tu_usuario@$IP "mysqldump -u root -pTuPassword tu_base_datos" > $Archivo

Write-Host "Backup completado en: $Archivo"
```

Este script está escrito en PowerShell (el terminal de comandos de Windows) y sirve para automatizar de forma remota la copia de seguridad de la base de datos MariaDB/MySQL alojada en tu máquina virtual Debian, guardando el archivo directamente en tu equipo Windows.

Variables iniciales (Configuración):

- \$IP: La dirección de red de tu máquina virtual Debian.
- \$Carpeta: La ruta en tu ordenador Windows donde quieres que se guarden los archivos generados.
- \$Archivo: Define el nombre del backup combinando la carpeta, el texto "backup\_" y la fecha y hora exacta actual (año-mes-día\_hora-minuto-segundo) para que ningún backup borre al anterior.

Comprobación de conexión:

- Hace un ping rápido (Test-Connection) a la máquina virtual. Si la máquina está apagada o no hay conexión, el script se cierra inmediatamente (exit) para evitar generar errores.

Creación del directorio:

- Revisa si la carpeta C:\Backups\_Debian existe en tu Windows. Si no existe, la crea automáticamente (New-Item) para que no falle el guardado.

Extracción remota por SSH y guardado:

- Se conecta por SSH a tu Debian de forma transparente.
- Ejecuta dentro de Debian el comando mysqldump para extraer la base de datos en texto plano.
- El símbolo > (redirección) toma todo ese texto que escupe la máquina virtual y lo escribe directamente en el archivo físico .sql de tu ordenador Windows.

Mensaje Final:

- Muestra un mensaje en la consola avisándote de que el proceso ha terminado y recordándote la ruta exacta donde se ha descargado el archivo.

Para que el script Backup\_LoveCode.ps1 se ejecute diariamente, se ha utilizado el Programador de Tareas de Windows siguiendo este procedimiento:

1. Creación de la tarea:
  - Abrir el "Programador de tareas" y seleccionar "Crear tarea básica".



- Asignar el nombre: Backup\_Diario\_LoveCode.
2. Programación del disparador:
- Seleccionar "Diariamente" y configurar la hora de ejecución preferida, estableciendo la periodicidad de repetición cada 1 día.
3. Configuración de la acción:
- Seleccionar "Iniciar un programa".
  - Programa o script: powershell.exe.
  - Argumentos: -ExecutionPolicy Bypass -File "C:\Ruta\TuScript\Backup\_LoveCode.ps1".

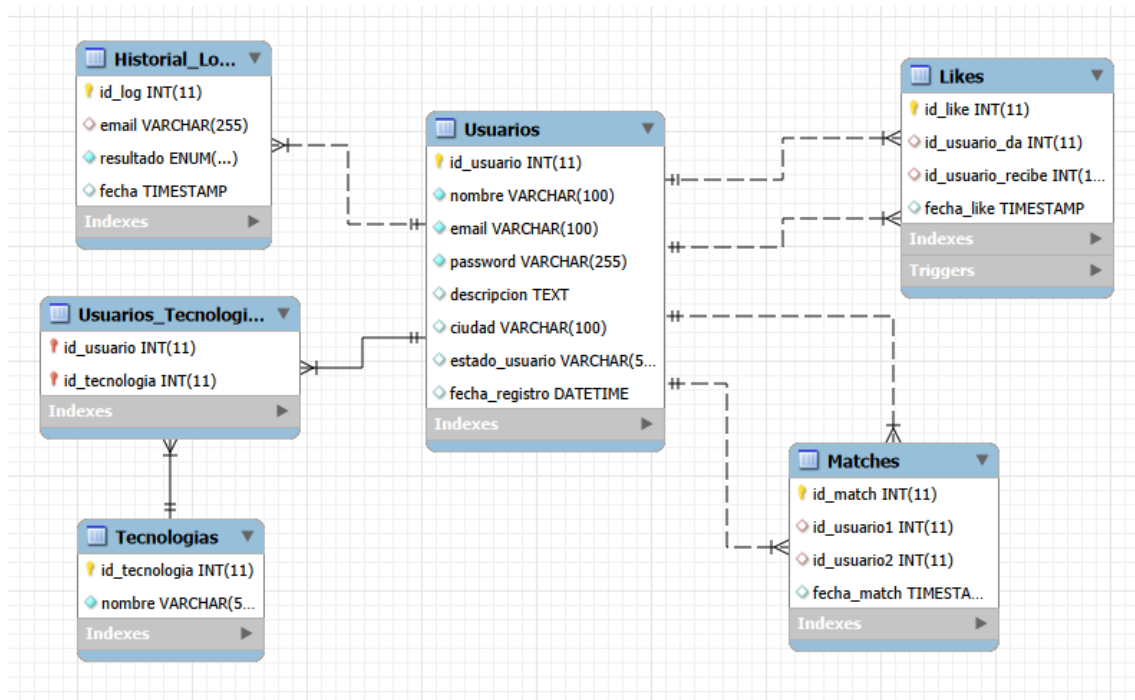
### 3.2.5 RESTAURACIÓN DEL BACKUP

Se utiliza la herramienta cliente de MySQL para inyectar el archivo de respaldo generado por el script de backup. El comando utilizado es: `mysql -u root -p tu_base_datos < C:\Backups_Debian\backup_YYYYMMDD_HHMMSS.sql`

## 4. BASES DE DATOS

La gestión de datos de LoveCode se realiza mediante MariaDB, un motor de base de datos relacional que corre dentro del servidor Debian. El diseño está pensado para soportar una red social, por lo que la base de datos lovecode\_db cuenta con tablas organizadas y relacionadas entre sí.

### 4.1 DIAGRAMA ENTIDAD-RELACIÓN



**El modelo relacional está compuesto por 6 tablas principales:**

- **Usuarios:** Es el núcleo del sistema. Almacena los datos personales (nombre, email, contraseña, ciudad, descripción) y cuenta con un sistema de borrado lógico mediante el campo estado\_usuario (activo/inactivo).

```
DROP TABLE IF EXISTS `Usuarios`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Usuarios` (
  `id_usuario` int(11) NOT NULL AUTO_INCREMENT,
  `nombre` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `descripcion` text DEFAULT NULL,
  `ciudad` varchar(100) DEFAULT NULL,
  `estado_usuario` varchar(50) DEFAULT 'activo',
  `fecha_registro` datetime DEFAULT current_timestamp(),
  PRIMARY KEY (`id_usuario`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=54 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla usuarios se puede destacar el campo único de email configurado como UNIQUE KEY para impedir de forma nativa que dos personas se registren con la misma cuenta de correo electrónico.

- **tecnologías:** Un catálogo que almacena los lenguajes de programación y herramientas disponibles (como Java, Python, SQL, CSS, etc.).

```
DROP TABLE IF EXISTS `Tecnologias`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Tecnologias` (
  `id_tecnologia` int(11) NOT NULL AUTO_INCREMENT,
  `nombre` varchar(50) NOT NULL,
  PRIMARY KEY (`id_tecnologia`),
  UNIQUE KEY `nombre` (`nombre`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla Tecnologías se puede destacar el campo “nombre” definido como UNIQUE KEY para evitar la duplicidad de lenguajes o herramientas en el catálogo del sistema (p. ej., que existan dos registros para "Java").

- **Usuarios\_Tecnologías:** Tabla intermedia (N:M) que relaciona a los usuarios con las tecnologías que conocen o les interesan.

```
DROP TABLE IF EXISTS `Usuarios_Tecnologias`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Usuarios_Tecnologias` (
  `id_usuario` int(11) NOT NULL,
  `id_tecnologia` int(11) NOT NULL,
  PRIMARY KEY (`id_usuario`,`id_tecnologia`),
  KEY `id_tecnologia` (`id_tecnologia`),
  CONSTRAINT `Usuarios_Tecnologias_ibfk_1` FOREIGN KEY (`id_usuario`) REFERENCES `Usuarios` (`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `Usuarios_Tecnologias_ibfk_2` FOREIGN KEY (`id_tecnologia`) REFERENCES `Tecnologias` (`id_tecnologia`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla Usuarios\_Tecnologias destaca la clave primaria compuesta, esta unión de id\_usuario e id\_tecnologia conforma la clave primaria, lo que impide físicamente que un usuario asigne la misma tecnología a su perfil más de una vez.

También la Integridad con ON DELETE CASCADE, configurado en ambas claves foráneas. Si un usuario se elimina del sistema de forma física, o si se borra una tecnología del catálogo general, sus asociaciones en esta tabla intermedia se limpian automáticamente sin dejar registros huérfanos.

- **Likes:** Registra de forma unidireccional cuando un usuario (id\_usuario\_da) muestra interés por otro (id\_usuario\_recibe).

```
DROP TABLE IF EXISTS `Likes`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Likes` (
  `id_like` int(11) NOT NULL AUTO_INCREMENT,
  `id_usuario_da` int(11) DEFAULT NULL,
  `id_usuario_recibe` int(11) DEFAULT NULL,
  `fecha_like` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id_like`),
  UNIQUE KEY `id_usuario_da` (`id_usuario_da`,`id_usuario_recibe`),
  KEY `id_usuario_recibe` (`id_usuario_recibe`),
  CONSTRAINT `Likes_ibfk_1` FOREIGN KEY (`id_usuario_da`) REFERENCES `Usuarios` (`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `Likes_ibfk_2` FOREIGN KEY (`id_usuario_recibe`) REFERENCES `Usuarios` (`id_usuario`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=59 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla likes destaca la restricción de unicidad cruzada, posee una clave única compuesta (UNIQUE KEY (id\_usuario\_da, id\_usuario\_recibe)). Esto restringe a nivel de base de datos que un desarrollador pueda darle Like más de una vez al mismo perfil.

También la integridad con ON DELETE CASCADE, si cualquiera de los dos usuarios involucrados (el que da o el que recibe el Like) es eliminado físicamente de la base de datos, el registro del Like se suprime automáticamente en cascada.

- **Matches:** Almacena las conexiones exitosas entre dos usuarios. Tiene restricciones para evitar duplicados sin importar el orden de los IDs (id\_usuario1, id\_usuario2).

## Memoria técnica Proyecto Intermodular

```
DROP TABLE IF EXISTS `Matches`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Matches` (
  `id_match` int(11) NOT NULL AUTO_INCREMENT,
  `id_usuario1` int(11) DEFAULT NULL,
  `id_usuario2` int(11) DEFAULT NULL,
  `fecha_match` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id_match`),
  UNIQUE KEY `id_usuario1` (`id_usuario1`,`id_usuario2`),
  KEY `id_usuario2` (`id_usuario2`),
  CONSTRAINT `Matches_ibfk_1` FOREIGN KEY (`id_usuario1`) REFERENCES `Usuarios` (`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `Matches_ibfk_2` FOREIGN KEY (`id_usuario2`) REFERENCES `Usuarios` (`id_usuario`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla matches destacamos la restricción de unicidad simétrica ya que cuenta con una UNIQUE KEY (id\_usuario1, id\_usuario2) para evitar que se duplique la conexión entre las dos mismas personas.

También la integridad con ON DELETE CASCADE: Vinculado en cascada a la tabla Usuarios. La eliminación de cualquiera de los dos miembros del Match purga de manera automática el registro de la relación.

- **Historial\_Logs:** Sistema de auditoría que registra eventos de los usuarios (como intentos de login) indicando si el resultado fue un 'EXITO' o un 'FALLO', junto con su marca de tiempo.

```
DROP TABLE IF EXISTS `Historial_Logs`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!40101 SET character_set_client = utf8mb4 */;
CREATE TABLE `Historial_Logs` (
  `id_log` int(11) NOT NULL AUTO_INCREMENT,
  `email` varchar(255) DEFAULT NULL,
  `resultado` enum('EXITO','FALLO') NOT NULL,
  `fecha` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id_log`),
  KEY `email` (`email`),
  CONSTRAINT `Historial_Logs_ibfk_1` FOREIGN KEY (`email`) REFERENCES `Usuarios` (`email`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_uca1400_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

De la tabla Historial\_Logs se puede destacar la restricción ON DELETE SET NULL, a diferencia de las anteriores, la clave foránea del campo email utiliza ON DELETE SET NULL. Si un usuario es eliminado de la plataforma, el registro de auditoría de sus inicios de sesión no se borra, sino que el campo se establece en NULL para conservar el histórico de accesos y la integridad estadística de los servidores.

También la restricción ENUM, el campo resultado está limitado estrictamente por restricción nativa a almacenar únicamente los valores 'EXITO' o 'FALLO'.

### Relaciones

El esquema implementa un fuerte control de cascadas (ON DELETE CASCADE) para mantener la base de datos limpia si se borra un usuario:

1. **Usuarios - Tecnologías (N:M):** Relación de muchos a muchos. Si se elimina un usuario o una tecnología, sus registros correspondientes en Usuarios\_Tecnologías se borran automáticamente.
2. **Usuarios - Likes (1:N):** Un usuario participa en varios "likes" (como emisor o receptor). Si el usuario desaparece, sus likes emitidos y recibidos se eliminan en cascada.
3. **Usuarios - Matches (1:N):** Un usuario puede tener múltiples "matches". Igual que en los Likes, existe un ON DELETE CASCADE hacia los dos usuarios implicados.
4. **Usuarios - Historial\_Logs (1:N):** En este caso, si se elimina el usuario, la foreign key asociada a su email se actualiza a NULL (ON DELETE SET NULL), permitiendo conservar el historial de eventos aunque el usuario ya no exista.

#### 4.2 Triggers (Disparadores)

- **tr\_after\_insert\_like:** Se activa automáticamente después de insertar un nuevo like. Comprueba si el like es mutuo y si ambos usuarios comparten al menos una tecnología en común. Si se cumplen ambas condiciones, crea automáticamente un registro en la tabla Matches.
- **tr\_after\_delete\_like:** Se ejecuta al eliminar un *like* y se encarga de borrar el match correspondiente en la tabla Matches si este existía.

#### 4.3 PROCEDIMIENTOS ALMACENADOS

El script incluye una extensa lista de procedimientos para abstraer la lógica de negocio directamente en la base de datos. Algunos de los principales son:

- Gestión de usuarios: crear\_usuario, borrar\_usuario, sp\_registrar\_usuario, sp\_buscar\_usuario\_por\_email, sp\_listar\_todos\_usuarios, sp\_listar\_usuarios\_dashboard.
- Gestión de tecnologías: sp\_asignar\_tecnologia\_usuario, sp\_obtener\_tecnologias\_usuario, sp\_listar\_tecnologias.
- Lógica de interacción (Likes/Matches): sp\_dar\_like, sp\_comprobar\_like\_mutuo, sp\_comparten\_tecnologia, sp\_crear\_match, sp\_comprobar\_si\_hay\_match, contar\_matches, sp\_listar\_likes\_dados, sp\_listar\_matches.

#### 4.4 USUARIOS

En el entorno de sistemas de LoveCode, se ha implementado una estructura de control de acceso basada en tres perfiles con distintos niveles de responsabilidad para garantizar la seguridad de la base de datos:

## Memoria técnica Proyecto Intermodular

**Usuario Administrador (administrador):** Es el perfil con mayores privilegios (ALL PRIVILEGES) sobre la base de datos lovecode\_db. Este usuario 'root' tiene todo el control.

**Usuario Desarrollador (desarrollador):** Este perfil está orientado a la fase de construcción y pruebas del backend. Tiene permisos para insertar, actualizar y consultar datos (DML).

**Usuario Lector (lector):** Es un perfil restringido únicamente a la consulta de información (SELECT). Se utiliza para tareas de auditoría o visualización simple, asegurando que bajo ninguna circunstancia se puedan modificar o eliminar los registros existentes de usuarios o tecnologías en el sistema.

```
MariaDB [(none)]> create database lovecode;
Query OK, 1 row affected (0,001 sec)

MariaDB [(none)]> use lovecode;
Database changed
MariaDB [lovecode]> CREATE USER 'administrador'@'localhost' IDENTIFIED BY 'administradorpablo';
Query OK, 0 rows affected (0,008 sec)

MariaDB [lovecode]> GRANT ALL PRIVILEGES ON LoveCode.* TO 'administrador'@'localhost' WITH GRANT OPTION;
Query OK, 0 rows affected (0,006 sec)

MariaDB [lovecode]> CREATE USER 'desarrollador'@'localhost' IDENTIFIED BY 'desarrolladorpablo';
Query OK, 0 rows affected (0,003 sec)

MariaDB [lovecode]> GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, INDEX, REFERENCES ON LoveCode.* TO 'desarrollador'@'localhost';
Query OK, 0 rows affected (0,002 sec)

MariaDB [lovecode]> CREATE USER 'lector'@'localhost' IDENTIFIED BY 'lectorpablo';
Query OK, 0 rows affected (0,004 sec)

MariaDB [lovecode]> GRANT SELECT ON LoveCode.* TO 'lector'@'localhost';
Query OK, 0 rows affected (0,005 sec)

MariaDB [lovecode]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)
```

Posteriormente se ha editado el “localhost” por “%” para que se puedan utilizar en una máquina remota que no sea la propia de debian donde esta alojada la BBDD.

Evidencias:

The first screenshot shows the command `SHOW GRANTS FOR 'administrador'@'%';` and the result grid displaying the grants for the 'administrador' user, including all privileges on the 'LoveCode' database.

The second screenshot shows the command `SHOW GRANTS FOR 'desarrollador'@'%';` and the result grid displaying the grants for the 'desarrollador' user, including SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, INDEX, and REFERENCES on the 'LoveCode' database.

The third screenshot shows the command `SHOW GRANTS FOR 'lector'@'%';` and the result grid displaying the grants for the 'lector' user, including SELECT on the 'LoveCode' database.

## 4.5 CONSULTAS SQL Y EVIDENCIAS

### Consulta 1

```
SELECT id_usuario, nombre, password
FROM Usuarios
WHERE email = 'pablo@pedron.com' AND estado_usuario = 'activo';
```

Procedimiento de origen: sp\_buscar\_usuario\_por\_email

Descripción: Es la consulta de autenticación. Busca en la tabla Usuarios los datos de acceso del programador que coincida con el email, asegurándose de que su cuenta no haya sido dada de baja (estado 'activo').

Evidencia:

```
1 CALL sp_buscar_usuario_por_email('pablo@pedron.com');
```

| Result Grid   Filter Rows:   Export:   Wrap Cell Content: |            |                    |                                                                 |
|-----------------------------------------------------------|------------|--------------------|-----------------------------------------------------------------|
|                                                           | id_usuario | nombre             | password                                                        |
| ▶                                                         | 34         | Pablo Pedrón López | \$2a\$10\$iv0sBSqRdg920WhzwzZ6UOZXpoRxfj3EV4APTVHQy2DCSvRH6VKL. |

### Consulta 2

```
SELECT u.id_usuario as id, u.nombre, u.email, u.ciudad, u.descripcion
FROM Usuarios u
INNER JOIN Matches m ON (u.id_usuario = m.id_usuario1 OR u.id_usuario = m.id_usuario2)
WHERE (m.id_usuario1 = 35 OR m.id_usuario2 = 35)
AND u.id_usuario != 35;
```

Procedimiento de origen: sp\_listar\_matches

Descripción: Esta consulta une la tabla de Usuarios con la de Matches. Busca las filas donde el usuario participa en la conexión y extrae los nombres y datos del otro desarrollador con el que hubo interés mutuo.

Evidencia:

```
1 call sp_listar_matches(35);
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

|   | id | nombre     | email           | ciudad   | descripcion   |
|---|----|------------|-----------------|----------|---------------|
| ▶ | 39 | Oscar Sasi | oscar@gmail.com | Vinalesa | Me gusta todo |

### Consulta 3

```
SELECT COUNT(*) AS total_matches
FROM Matches
WHERE id_usuario1 = 34 OR id_usuario2 = 34;
```

Procedimiento de origen: contar\_matches

Descripción: Una consulta simple que utiliza la función COUNT(\*) sobre la tabla Matches para calcular y devolver en un solo número cuántas conexiones o parejas totales ha consolidado el usuario en la aplicación.

Evidencia:

```
1 call contar_matches(35);
```

| Result Grid   |   | Filter Rows: | Export: | Wrap Cell Content: |
|---------------|---|--------------|---------|--------------------|
| total_matches |   |              |         |                    |
| 1             | 1 |              |         |                    |

### Consulta 4

```
SELECT id_usuario as id, nombre, email, ciudad, descripcion
FROM Usuarios
WHERE id_usuario != 34
AND estado_usuario = 'activo'
AND id_usuario NOT IN (SELECT id_usuario_recibe FROM Likes WHERE id_usuario_da = 34);
```

Procedimiento de origen: sp\_listar\_usuarios\_dashboard

Descripción: Es la consulta que genera el feed de perfiles. Selecciona usuarios activos excluyéndose a uno mismo, y utiliza un filtrado (NOT IN) para descartar de la lista a todas las personas a las que ya se les dio un Like.

Evidencia:

```
1 call sp_listar_usuarios_dashboard(34);
```

| Result Grid |              | Filter Rows:        | Export:   | Wrap Cell Content:                              |
|-------------|--------------|---------------------|-----------|-------------------------------------------------|
| id          | nombre       | email               | ciudad    | descripcion                                     |
| 38          | Sergio Chill | sergio@gmail.com    | Valencia  | Me gusta JavaScript y XML                       |
| 40          | Jose Luis    | jose Luis@gmail.com | Torrente  | Fan de Java                                     |
| 42          | Esteban      | esteban@gmail.com   | Valencia  | Soy el dios del SQL                             |
| 43          | Pedro        | pedro@gmail.com     | Valencia  | Me gusta el frontend                            |
| 45          | Sara López   | sara@gmail.com      | Madrid    | Me gusta Java y Python                          |
| 52          | Marta G      | marta@gmail.com     | Barcelona | Me gusta Java, JavaScript y hacer consultas SQL |



## Consulta 5

```
SELECT t.id_tecnologia, t.nombre
FROM Technologies t
INNER JOIN Usuarios_Tecnologias ut ON t.id_tecnologia = ut.id_tecnologia
WHERE ut.id_usuario = 34
ORDER BY t.nombre;
```

Procedimiento de origen: sp\_obtener\_tecnologias\_usuario

Descripción: Consulta con INNER JOIN que conecta las tablas de Tecnologías y la intermedia Usuarios\_Tecnologías. Sirve para traer al frente ordenados alfabéticamente los lenguajes de programación específicos añadidos al perfil del usuario.

Evidencia:

```
1 call sp_obtener_tecnologias_usuario('35');
```

| Result Grid   |        | Filter Rows: | Export: | Wrap Cell Content: |
|---------------|--------|--------------|---------|--------------------|
| id_tecnologia | nombre |              |         |                    |
| 3             | CSS    |              |         |                    |
| 2             | HTML   |              |         |                    |

## Consulta 6

```
SELECT email, resultado, fecha
FROM Historial_Logs
ORDER BY fecha DESC
LIMIT 5;
```

Descripción: Esta consulta accede a la tabla de auditoría para extraer los 5 registros de inicio de sesión más recientes. Es ideal para verificar la actividad de seguridad del sistema y comprobar quién ha intentado acceder recientemente y si tuvieron éxito o fallo.

Evidencia

```
1 • SELECT email, resultado, fecha
2 FROM Historial_Logs
3 ORDER BY fecha DESC
4 LIMIT 5;
```

| Result Grid      |           |                     | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |
|------------------|-----------|---------------------|--------------|---------|--------------------|-------------|
| email            | resultado | fecha               |              |         |                    |             |
| oscar@gmail.com  | EXITO     | 2026-05-29 08:47:16 |              |         |                    |             |
| alba@gmail.com   | EXITO     | 2026-05-29 08:42:57 |              |         |                    |             |
| alba@gmail.com   | EXITO     | 2026-05-29 08:42:41 |              |         |                    |             |
| alba@gmail.com   | EXITO     | 2026-05-29 08:42:39 |              |         |                    |             |
| pablo@pedron.com | EXITO     | 2026-05-29 08:39:57 |              |         |                    |             |

## Consulta 7

```
SELECT nombre, descripcion
FROM Usuarios
WHERE ciudad != 'Valencia'
AND estado_usuario = 'activo';
```

Descripción: Filtra a todos los programadores activos que no residen en Valencia. Esta consulta sirve para segmentar la base de datos y realizar búsquedas de usuarios en otras regiones geográficas, excluyendo los perfiles locales.

Evidencia:

```
1 • SELECT nombre, descripcion
2 FROM Usuarios
3 WHERE ciudad != 'Valencia'
4 AND estado_usuario = 'activo';
```

| nombre     | descripcion                                     |
|------------|-------------------------------------------------|
| Oscar Sasi | Me gusta todo                                   |
| Jose Luis  | Fan de Java                                     |
| Sara López | Me gusta Java y Python                          |
| Marta G    | Me gusta Java, JavaScript y hacer consultas SQL |

## Consulta 8

```
SELECT id_tecnologia, COUNT(*) AS numero_de_usuarios
FROM Usuarios_Tecnologias
GROUP BY id_tecnologia
HAVING numero_de_usuarios > 2;
```

Descripción: Realiza una agrupación por tecnología y filtra únicamente aquellas que están siendo utilizadas por más de 2 usuarios en la plataforma. Es una consulta muy útil para detectar qué lenguajes son los más populares o tienen mayor adopción dentro de LoveCode.

Evidencia:

```
1 • SELECT id_tecnologia, COUNT(*) AS numero_de_usuarios
2 FROM Usuarios_Tecnologias
3 GROUP BY id_tecnologia
4 HAVING numero_de_usuarios > 2;
```

| id_tecnologia | numero_de_usuarios |
|---------------|--------------------|
| 1             | 4                  |
| 3             | 4                  |
| 4             | 4                  |
| 5             | 5                  |
| 6             | 4                  |

## 5. PROGRAMACIÓN

En este apartado se describe la lógica implementada en el backend de LoveCode, desarrollado en Java 21 con Spring Boot 3.2.4 y gestionado con Apache Maven. El servidor expone una API REST que el frontend consume mediante fetch(), devolviendo siempre respuestas en formato JSON.

### 5.1 CONEXIÓN CON LA BASE DE DATOS

La conexión a MariaDB se gestiona en la clase ConexionDB.java mediante el driver JDBC oficial (mariadb-java-client). El método estático conectar() abre una nueva conexión en cada llamada usando DriverManager.getConnection():

```
private static final String URL = "jdbc:mariadb://192.168.204.129:3306/lovecode_db";
private static final String USUARIO = "administrador";

public static Connection conectar() {
    return DriverManager.getConnection(URL, USUARIO, PASSWORD);
}
```

La IP 192.168.204.129 corresponde a la máquina virtual Debian configurada en modo Host-Only. La BD escucha en el puerto 3306 con bind-address configurado a 0.0.0.0 para aceptar conexiones externas.

Todos los accesos a datos siguen el patrón DAO (Data Access Object): cada DAO abre su propia conexión, la usa y la cierra automáticamente mediante try-with-resources, evitando fugas de conexión:

```
String sql = "{call sp_registrar_usuario(?, ?, ?, ?, ?, ?)}";
try (Connection conn = ConexionDB.conectar();
    CallableStatement cs = conn.prepareCall(sql)) {
    cs.setString(1, u.getNombre());
    cs.registerOutParameter(6, Types.INTEGER); // Param OUT: ID generado
    cs.execute();
    return cs.getInt(6);
}
```

Los DAOs utilizan CallableStatement para ejecutar stored procedures de MariaDB, lo que mantiene el SQL encapsulado en la base de datos y el código Java desacoplado de la lógica SQL.

## 5.2 OPERACIONES QUE REALIZA EL SERVIDOR

El backend expone los siguientes endpoints REST, organizados por controlador:

| Controlador          | Método + Ruta             | Operación                                                                                                      |
|----------------------|---------------------------|----------------------------------------------------------------------------------------------------------------|
| AuthController       | POST /api/auth/registro   | Registra un nuevo usuario: hashea la contraseña con BCrypt, llama a sp_registrar_usuario y asigna tecnologías. |
| AuthController       | POST /api/auth/login      | Autentica al usuario: busca por email, compara hash BCrypt y registra el intento en Historial_Logs.            |
| UsuarioController    | GET /api/usuarios         | Devuelve todos los usuarios activos. Si se pasa ?idUsuario=X, filtra los ya vistos/likeados para el dashboard. |
| LikeController       | POST /api/likes           | Registra un like. Tras la inserción, consulta si el trigger generó un match y lo informa en la respuesta.      |
| LikeController       | DELETE /api/likes         | Elimina un like. El trigger tr_after_delete_like borra el match correspondiente automáticamente.               |
| LikeController       | GET /api/likes/dados/{id} | Devuelve la lista de usuarios a los que el usuario ha dado like.                                               |
| MatchController      | GET /api/matches/{id}     | Devuelve la lista de matches del usuario logueado.                                                             |
| TecnologiaController | GET /api/tecnologias      | Devuelve el catálogo de tecnologías disponibles para el formulario de registro.                                |

Todos los controladores usan @CrossOrigin(origins = "\*") para permitir que el navegador acepte las respuestas del servidor, y devuelven ResponseEntity<?> con códigos HTTP apropiados (200, 400, 401, 500).

## 5.3 REGISTRO Y LOGIN

### Registro de nuevo usuario

El flujo completo de registro es el siguiente:

- El frontend envía un JSON con nombre, email, contraseña, ciudad, descripción y la lista de IDs de tecnologías seleccionadas.
- AuthController recibe el objeto Usuario deserializado automáticamente por Spring Boot.
- Se encripta la contraseña con BCrypt antes de persistirla: passwordEncoder.encode(u.getPassword()). Nunca se guarda la contraseña en texto plano.
- UsuarioDAO.registrar() ejecuta el stored procedure sp\_registrar\_usuario (5 params IN + 1 param OUT con el ID generado). Si el email ya existe, el procedimiento lanza un error SIGNAL que se captura como excepción.
- Si el registro es exitoso y el usuario seleccionó tecnologías, TecnologiaDAO.asignarTecnologias() ejecuta en batch el stored procedure sp\_asignar\_tecnologia\_usuario para cada tecnología elegida.
- Se devuelve { "mensaje": "Usuario registrado correctamente" } con HTTP 200, o un error con HTTP 500 si algo falla.

### Login de usuario existente

El flujo de autenticación es el siguiente:

- El frontend envía un JSON con email y contraseña en texto plano.
- UsuarioDAO.buscarPorEmail() ejecuta sp\_buscar\_usuario\_por\_email, que solo devuelve usuarios con estado\_usuario = 'activo'. Las cuentas desactivadas no pueden iniciar sesión.
- Si se encuentra el usuario, BCrypt compara la contraseña enviada con el hash guardado: passwordEncoder.matches(passPlano, hashGuardado).
- En ambos casos (éxito o fallo), se registra el evento en Historial\_Logs mediante UsuarioDAO.registrarLog(email, "EXITO" / "FALLO"), incluyendo intentos con emails no existentes.
- Si el login es correcto, el servidor devuelve { "status": "ok", "nombre": ..., "id": ... }. El frontend guarda estos datos en localStorage (lovecode\_user y lovecode\_user\_id) para mantener la sesión.
- Si las credenciales son incorrectas, se devuelve HTTP 401 con { "error": "Credenciales incorrectas" }.

### 5.4 ALMACENAMIENTO DE LIKES

Los likes se gestionan a través de LikeController y LikeDAO, apoyándose en los stored procedures y triggers de la base de datos:

#### Dar un like

- El frontend envía { idEmisor, idReceptor } en el body de la petición POST /api/likes.
- El controlador valida que el emisor no sea el mismo que el receptor (no se puede dar like a uno mismo).
- LikeDAO.darLike() ejecuta el stored procedure sp\_dar\_like, que inserta el registro en la tabla Likes. La clave única compuesta (id\_usuario\_da, id\_usuario\_recibe) impide likes duplicados.
- Tras la inserción, el trigger tr\_after\_insert\_like se dispara automáticamente en la BD: comprueba si existe like mutuo y si los dos usuarios comparten al menos una tecnología. Si ambas condiciones se cumplen, el trigger crea el match en la tabla Matches.
- El backend consulta si hay match llamando a sp\_comprobar\_si\_hay\_match y devuelve al frontend { "mensaje": "Like registrado", "match": true/false }.
- Si ya existía el like (entrada duplicada), se captura la excepción y se devuelve HTTP 400 con mensaje de error.

#### Quitar un like

- El frontend envía { idEmisor, idReceptor } en el body de la petición DELETE /api/likes.
- LikeDAO.quitarLike() ejecuta un DELETE directo sobre la tabla Likes.

## Memoria técnica Proyecto Intermodular

- El trigger `tr_after_delete_like` se dispara automáticamente y elimina el match correspondiente en la tabla `Matches` (si existía), manteniendo la consistencia de datos sin lógica adicional en el backend.

### Consultar likes dados

El endpoint `GET /api/likes/dados/{idUsuario}` devuelve la lista de usuarios a los que el usuario ha dado like, enriquecida con sus tecnologías mediante `TecnologiaDAO.rellenarTecnologias()`. Esta función realiza una consulta adicional por cada usuario para obtener sus tecnologías y las añade al mapa de datos como campo `tecnologias`.

## 5.5 EVIDENCIAS DE FUNCIONAMIENTO

Se crea usuario

**LOVEC3DE**  
Crea tu usuario

String nombre;  
Pepe López

String email;  
pepelopez@gmail.com

String password;  
\*\*\*\*

String ciudad;  
Valencia

String descripcion;  
Me gustan todas las tecnologías

List<Tecnologia> misTecnologias;

☒ C# ☒ ASP ☒ Java ☒ JavaScript  
☒ Python ☒ PHP ☒ JSP

**Ejecutar (Crear Cuenta)**

[¿Ya estás en la base de datos? Log In](#)

```
1 • select * from Usuarios;
```

|    |            |                     |                                                           |                                 |          |        |                     |
|----|------------|---------------------|-----------------------------------------------------------|---------------------------------|----------|--------|---------------------|
| 54 | Pepe López | pepelopez@gmail.com | \$2a\$10\$p5eZ6dHxvAJUjD3k.zU8p3UWRyHtH9eJahViyF4GvSB756s | Me gustan todas las tecnologías | Valencia | activo | 2026-05-31 21:09:57 |
|----|------------|---------------------|-----------------------------------------------------------|---------------------------------|----------|--------|---------------------|

Se loguea

**LOVEC3DE**  
Inicia sesión en tu servidor

String email;  
pepelopez@gmail.com

String password;  
\*\*\*\*

**Ejecutar (Entrar)**

[¿No tienes cuenta? Registrate](#)

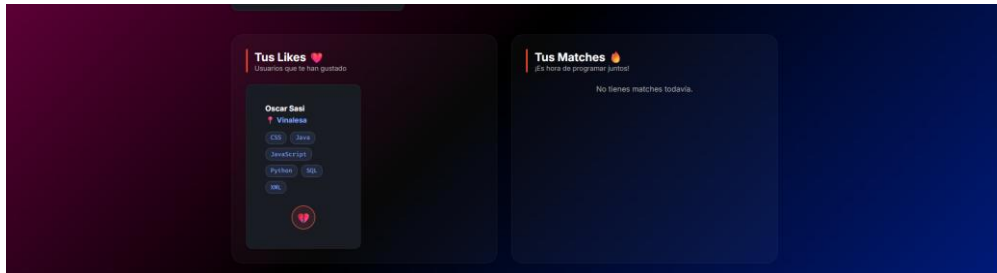
```
1 • select * from Historial_Logs;
```

|    |                     |       |                     |
|----|---------------------|-------|---------------------|
| 24 | pepelopez@gmail.com | EXITO | 2026-05-31 21:09:57 |
|----|---------------------|-------|---------------------|

## Memoria técnica Proyecto Intermodular



Damos like a otro usuario



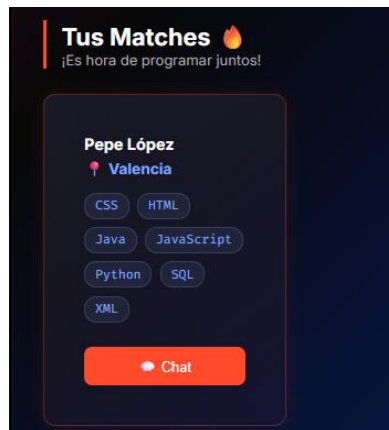
```
1 • select * from Likes where id_usuario_da='54';
```

|   | id_like | id_usuario_da | id_usuario_recibe | fecha_like          |
|---|---------|---------------|-------------------|---------------------|
| ▶ | 63      | 54            | 39                | 2026-05-31 21:13:23 |

Nos logueamos en la otra cuenta



Se le da like a “Pepe López” desde la otra cuenta a la que el ha dado like, como coinciden en tecnologías y tienen un like ambos se crea el match



```
1 • select * from Matches;
```

|   | id_match | id_usuario1 | id_usuario2 | fecha_match         |
|---|----------|-------------|-------------|---------------------|
| ▶ | 7        | 39          | 54          | 2026-05-31 21:19:16 |

Si quitamos el like desaparecerá el match

|   | id_match | id_usuario1 | id_usuario2 | fecha_match |
|---|----------|-------------|-------------|-------------|
| * | NULL     | NULL        | NULL        | NULL        |

## 6. LENGUAJE DE MARCAS

El frontend esta compuesto por cuatro vistas principales, desarrolladas con HTML5, CSS3 y JavaScript vanilla, sin ningun framework adicional. Cada vista tiene su propio archivo HTML y CSS, compartiendo la logica de autenticacion a traves de un archivo JavaScript comun.

### 6.1 ESTRUCTURA DEL FRONTEND

| Archivo               | Tipo       | Funcion                                                                     |
|-----------------------|------------|-----------------------------------------------------------------------------|
| <b>index.html</b>     | HTML5      | Pagina de inicio publica. Muestra perfiles de ejemplo y CTA de registro.    |
| <b>index.css</b>      | CSS3       | Estilos de la landing page: navbar, hero, grid de perfiles, footer.         |
| <b>login.html</b>     | HTML5      | Formulario de inicio de sesion con tema de codigo.                          |
| <b>login.css</b>      | CSS3       | Estilos de la pantalla de login: tarjeta centrada con borde gradiente.      |
| <b>register.html</b>  | HTML5      | Formulario de registro con seleccion dinamica de tecnologias.               |
| <b>register.css</b>   | CSS3       | Estilos del formulario de registro incluida la rejilla de tecnologias.      |
| <b>dashboard.html</b> | HTML5      | Vista principal autenticada: perfiles, likes dados y matches.               |
| <b>dashboard.css</b>  | CSS3       | Estilos del dashboard: secciones secundarias, tarjetas match, animaciones.  |
| <b>Auth.js</b>        | JavaScript | Logica de registro, login y carga de tecnologias desde la API.              |
| <b>dashboard.js</b>   | JavaScript | Logica del dashboard: cargar perfiles, dar likes, matches y notificaciones. |



## 6.2 TECNOLOGÍAS

### HTML5

Se ha evitado el uso de etiquetas genéricas (<div>), sustituyéndolas por elementos que mejoran la accesibilidad y el posicionamiento:

- <header> y <nav> para la gestión de menús y la marca de la app.
- <main> para aislar el contenedor del perfil principal a evaluar.
- <section> y <aside> para diferenciar el flujo de perfiles de la barra lateral que contiene la lista de *Matches* interactivos.
- <form> con atributos nativos de validación (required, type="email") para asegurar la integridad inicial de los datos antes de enviarlos al sistema.

### CSS3

El apartado estético explota las capacidades modernas del estándar CSS3 sin depender de frameworks pesados:

- **Custom Properties (Variables CSS):** Centralización de la paleta de colores del *Dark Mode* (--bg-primary: #121212, --accent-color: #61afef, --neon-heart: #e06c75) para permitir cambios de temas globales de manera ágil.
- **Flexbox y CSS Grid:** Implementación de un diseño adaptable (Responsive Design). CSS Grid organiza la arquitectura general de la app (división entre el feed central y la barra lateral de matches), mientras que Flexbox alinea los botones de control y las etiquetas (badges) de las tecnologías del programador.
- **Efectos Visuales:** Uso de (:hover, :focus-within), transiciones suaves (transition: transform 0.3s ease) y sombras difuminadas (*glassmorphism*).

### JavaScript

La gran incorporación del frontend es la programación reactiva mediante JavaScript puro (Vanilla JS), estructurado en funciones modulares y asíncronas:

- **Manipulación Dinámica del DOM:** Al hacer clic en los botones de interacción ("Aceptar código" / "Generar Syntax Error"), JavaScript intercepta el evento, ejecuta una animación de salida en la tarjeta de perfil actual y renderiza el siguiente perfil disponible sin necesidad de recargar la página completa.
- **Programación Asíncrona (Fetch API):** Uso de estructuras async/await para realizar llamadas en segundo plano a los endpoints del Backend en Java.

Esto permite enviar un *Like* a la base de datos o comprobar si se ha producido un *Match* instantáneo de forma transparente para el usuario.

- **Validación Local de Formularios:** Control en tiempo real durante el registro de usuarios, verificando que los formatos de email sean válidos y proporcionando *feedback* visual inmediato mediante clases dinámicas de CSS.

## 7. ENTORNOS DE DESARROLLO

Para la construcción de LoveCode, se ha configurado un ecosistema de desarrollo profesional que permite la integración de tecnologías de backend, frontend y sistemas. El entorno se ha diseñado para ser escalable y facilitar el trabajo con la arquitectura de capas elegida.

### 7.1 CONTROL DE VERSIONES

La gestión del código fuente de **LoveCode** se basa en un sistema de control de versiones, lo que garantiza la integridad del proyecto y permite un desarrollo organizado y reversible.

- **Git y GitHub:** Se utiliza **Git** para el seguimiento local de los cambios y **GitHub** como plataforma de alojamiento remoto. Esto no solo funciona como un respaldo en la nube, sino que permite mantener un historial detallado de cada modificación realizada en el backend, el frontend y la base de datos.
- **Uso de Ramas (Branching):** Se ha implementado una metodología de trabajo basada en ramas para aislar las funcionalidades en desarrollo:
  - **Rama main:** Es la rama principal y estable del proyecto. Solo contiene código que ha sido verificado y que es totalmente funcional (como la conexión base y los formularios actuales).
  - **Ramas de Funcionalidad (feature branches):** Para cada nueva mejora (por ejemplo, el diseño del login o la lógica de encriptación), se crean ramas independientes. Esto permite experimentar y realizar cambios profundos sin poner en riesgo la estabilidad de la rama principal.
  - **Fusión (Merge):** Una vez que una funcionalidad está terminada y testeada, se integra en la rama main, asegurando que el repositorio siempre mantenga una versión operativa del software.

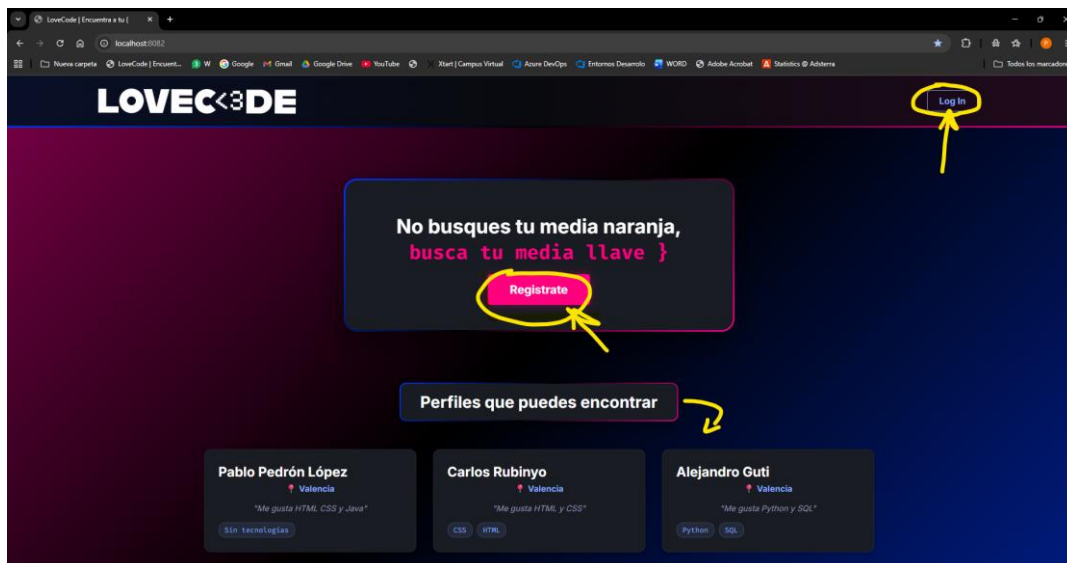
### 7.2 SOFTWARE UTILIZADO

Para el desarrollo integral de la aplicación, se ha seleccionado un conjunto de herramientas de software que cubren todas las necesidades del ciclo de vida del desarrollo:

- Desarrollo de Código (IDEs): Visual Studio Code
- Gestión de Dependencias: Apache Maven, que a través del archivo pom.xml centraliza y automatiza la descarga de las librerías necesarias (MariaDB Connector, SLF4J...).
- Gestión de Base de Datos: \* MariaDB: Como motor de base de datos relacional.
  - MySQL Workbench : Herramientas gráficas para el diseño de diagramas Entidad-Relación, ejecución de scripts SQL y monitorización de datos.
- Virtualización y Sistemas: VMware Workstation para ejecutar el servidor Debian 12 alojando la base de datos.

### 8. FUNCIONAMIENTO DEL PROYECTO

Al acceder a la dirección web de la aplicación, el usuario es recibido por la pantalla principal de **LoveCode**, en esta ventana, el nuevo visitante o el usuario ya registrado dispone de dos opciones, pulsar sobre el botón de inicio de sesión si ya dispone de una cuenta activa, o clicar en el botón de registro para comenzar el proceso de alta en la plataforma.



Para formar parte de la comunidad, el nuevo usuario debe rellenar formulario de registro con sus datos personales. Se solicita la inserción de su nombre completo, email de contacto, contraseña de seguridad, ciudad y una breve descripción así

## Memoria técnica Proyecto Intermodular

como sus tecnologías favoritas; tras pulsar el botón "Crear Cuenta", los datos se procesan de forma inmediata para crear su nuevo perfil.

The screenshot shows a web browser window with the URL 'localhost:8082/register.html'. The page has a dark blue background with a purple gradient on the left. A central white card contains the 'LOVECODE' logo and the text 'Crea tu usuario'. Below this, there are several input fields, each with a corresponding label and a yellow arrow pointing to it: 'String nombre;' (labeled 'Nombre completo'), 'String email;' (labeled 'developer@lovecode.com'), 'String password;' (labeled '.....'), 'String ciudad;' (labeled 'Valencia, Madrid ...'), 'String descripcion;' (labeled 'Cuéntanos algo sobre ti ...'), and a section for 'List<Tecnologias>: misTecnologias:' with checkboxes for 'JS', 'HTML', 'Java', 'JavaScript', 'Python', 'SQL', and 'PHP'. At the bottom of the card is a red button labeled 'Ejecutar (Crear Cuenta)'. Below the button, there is a link that says '¿Ya estás en la base de datos? Log In'.

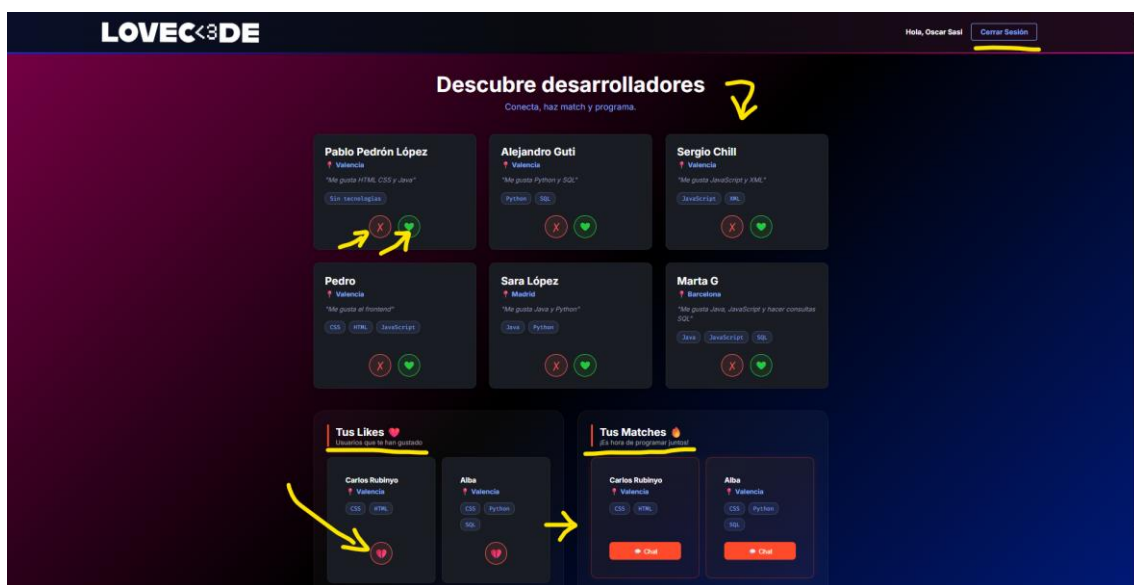
Cuando el usuario inicia sesión, el sistema muestra un formulario donde debe ingresar su correo electrónico y contraseña. Luego, al hacer clic en “Iniciar Sesión”, el sistema verifica sus datos y le permite acceder a la plataforma.

The screenshot shows a web browser window with the URL 'localhost:8082/login.html'. The page has a dark blue background with a purple gradient on the left. A central white card contains the 'LOVECODE' logo and the text 'Inicia sesión en tu servidor'. Below this, there are two input fields, each with a corresponding label and a yellow arrow pointing to it: 'String email;' (labeled 'mi@email.com') and 'String password;' (labeled '.....'). At the bottom of the card is a red button labeled 'Ejecutar (Entrar)'. Below the button, there is a link that says '¿No tienes cuenta? Registrate'.

Una vez que entramos a nuestra cuenta, llegamos a esta pantalla principal, que es la zona privada de **LoveCode**. Desde aquí podemos manejar toda nuestra actividad de forma muy intuitiva gracias a las funciones integradas en el panel:

- **Dar Likes:** En el centro de la pantalla nos irán apareciendo perfiles de otros desarrolladores. Podemos pulsar el botón de "Like" para mostrar interés en trabajar o conectar con ellos.

- **Quitar Likes:** Si cambiamos de opinión, el sistema también nos permite deshacer la acción en el apartado “Tus Likes”.
- **Apartado de Matches:** Tenemos una sección de matches, veremos una lista con todas aquellas personas con las que el interés ha sido mutuo (es decir, que nosotros les hemos dado like y ellos a nosotros también) y tengamos tecnologías en común.
- **Cerrar Sesión:** Cuando hayamos terminado, solo hay que pulsar el botón de "Cerrar Sesión" para salir de forma segura de nuestra cuenta, lo que nos devolverá automáticamente a la pantalla de entrada del proyecto.



## 9. PROBLEMAS ENCONTRADOS Y SOLUCIONES

**Problema de conexión por seguridad de red (CORS):** Al intentar comunicar los formularios del cliente (login.html y register.html) con la API de Spring Boot, el navegador web bloqueaba las peticiones asíncronas lanzadas por Auth.js debido a la política de seguridad del mismo origen (Cross-Origin Resource Sharing).

- **Solución:** Se consultó la documentación de desarrollo web de Mozilla Developer Network (s.f.) y se configuró la anotación `@CrossOrigin(origins = "*")` en los controladores Java, permitiendo el intercambio seguro de datos JSON entre el puerto del cliente y el servidor backend.

**Problema con la pérdida de rutinas en las copias de seguridad:** Al realizar las primeras exportaciones de la base de datos lovecode\_db mediante la consola de comandos de Linux, se detectó que el archivo .sql resultante omitía por completo los procedimientos almacenados y las funciones del historial de auditoría.

- *Solución:* Basándose en la documentación técnica de MariaDB Foundation (s.f.), se modificó el comando de volcado añadiendo el parámetro `--routines` (`mariadb-dump -p --routines lovecode_db > backup.sql`), logrando que el script final empaquetara de manera íntegra toda la estructura de datos y su lógica interna.

## 10. CONCLUSIONES

El desarrollo del proyecto **LoveCode** ha permitido consolidar de forma práctica y transversal los conocimientos adquiridos durante este primer curso de Desarrollo de Aplicaciones Web:

- Se ha logrado diseñar e implementar una arquitectura web completa y funcional utilizando un backend estructurado en Java con Spring Boot, una base de datos relacional MariaDB y un frontend dinámico basado en estándares nativos (HTML5, CSS3 y JavaScript Asíncrono).
- La implementación de lógica avanzada dentro del propio motor de base de datos como el uso de triggers automáticos para la auditoría del sistema ha demostrado ser una solución eficiente para descargar de tareas repetitivas al servidor de aplicaciones.
- El uso de herramientas de automatización, como scripts de comandos para arrancar la infraestructura y sistemas de control de versiones, ha evidenciado la importancia de optimizar los entornos de desarrollo para agilizar el despliegue y las pruebas de software.
- Como conclusión final, este proyecto sienta unas bases sólidas de programación orientada a objetos, gestión de redes y manipulación del DOM, esenciales para afrontar el desarrollo de aplicaciones de software más complejas en el siguiente año formativo.

## 11. BIBLIOGRAFÍA Y WEBGRAFÍA

Modo Debug. (s.f.). *Java Básico* [Lista de reproducción]. YouTube.

[https://www.youtube.com/playlist?list=PLG1qdjD\\_\\_qH6ULjW5iN8E45m5nkaCNbUuUu](https://www.youtube.com/playlist?list=PLG1qdjD__qH6ULjW5iN8E45m5nkaCNbUuUu)

DataCamp. (s.f.). Programación Orientada a Objetos (POO) en Java: Conceptos, ejemplos y más. <https://www.datacamp.com/es/tutorial/oop-in-java>

Spring by VMware. (s.f.). *Spring Boot Reference Guide*.  
<https://spring.io/projects/spring-boot>

MariaDB Foundation. (s.f.). *MariaDB Server Documentation*.  
<https://mariadb.com/kb/en/documentation/>

## Memoria técnica Proyecto Intermodular

Mozilla Developer Network. (s.f.). *MDN Web Docs*. <https://developer.mozilla.org/>

Oracle Corporation. (s.f.). *Java SE Documentation*.  
<https://docs.oracle.com/en/java/>